



The Papyrus Handbook

A guide to Markdown books with Papyrus

milon/papyrus

The Papyrus Handbook

Markdown to Book — PDF, EPUB, HTML, and KDP

Papyrus

Copyright © 2026 Papyrus.

This handbook is the companion guide for **milon/papyrus** — a PHP CLI that turns a Markdown book project into PDF, EPUB, HTML, and Amazon KDP exports.

Table of Contents

Introduction	10
What you get	11
Project shape	12
Next steps	13
Install and project layout	14
Requirements	15
Install per project	16
Install globally	17
Scaffold a book	18
Layout	19

Configuration	20
Identity	21
Page size and margins	22
Table of contents	23
Cover images	24
Running header	25
Fonts	26
Faces	27
Script rules	27
Break level	29
CommonMark hook	30
Writing content	31

Front matter	32
Markdown features	33
Emphasis and code	33
Callouts	33
Page breaks	34
Mermaid	34
PHP fence linting	36
Themes and assets	37
PDF themes	38
HTML theme	39
EPUB styles	40
Covers and fonts	41
Building exports	42

Individual formats	43
Watch mode	44
Caching	45
Vendor noise	46
Samples and KDP	47
Sample PDF	48
Amazon KDP	49
Migration and CI	51
Migrating from ibis-next	52
Continuous integration	53
Programmatic use	54

Checklist before release 55

Command reference 56

Introduction

Papyrus is a Composer package and CLI for authors who write books in Markdown and need print-ready PDF, EPUB, single-file HTML, and Amazon KDP outputs from one project layout.

This handbook is itself a Papyrus book. The source lives in [examples/the-papyrus-handbook/](#); rendered PDF and HTML are checked into [docs/](#) so you can read without building.

What you get

Format	Typical use
PDF	Print interiors, digital reading, sample PDFs
EPUB	Stores, e-readers, KDP Kindle upload
HTML	Online reading, sharing a single file
KDP	Kindle EPUB, print PDF, cover export, metadata JSON

Project shape

Every book is a directory with:

- `papyrus.php` — title, trim, themes, Mermaid, sample ranges, KDP
- `content/` — Markdown chapters (natural filename order)
- `assets/` — PDF themes, HTML theme, EPUB CSS, covers, fonts
- `export/` — build outputs (usually gitignored)

Next steps

Install Papyrus, scaffold or copy this sample, run `papyrus doctor`, then `papyrus build`. The chapters that follow cover each step in detail.

Install and project layout

Requirements

- PHP 8.2+ with extensions `dom`, `gd`, `mbstring`, `zip`, and `zlib`
- Composer
- Optional: Node.js and `@mermaid-js/mermaid-cli` (`mmdc`) when Mermaid is enabled

Install per project

In your book repository:

```
composer require milon/papyrus
```

```
vendor/bin/papyrus --version  
vendor/bin/papyrus init  
vendor/bin/papyrus doctor
```

Wire Composer scripts:

```
{  
  "scripts": {  
    "build": "papyrus build",  
    "build:pdf": "papyrus build:pdf --theme light,dark",  
    "build:epub": "papyrus build:epub",  
    "build:html": "papyrus build:html",  
    "build:sample": "papyrus build:sample",  
    "build:kdp": "papyrus kdp",  
    "doctor": "papyrus doctor"  
  }  
}
```


Install globally

```
composer global require milon/papyrus  
export PATH="$(composer global config bin-dir --absolute):$PATH"  
papyrus list
```

Scaffold a book

```
papyrus init  
papyrus init -d my-book
```

`init` creates `papyrus.php`, `content/`, and `assets/`. Use `--force` / `-f` to overwrite.

Layout

Path	Role
<code>papyrus.php</code>	Book settings
<code>content/</code>	Markdown chapters
<code>assets/</code>	Themes, CSS, covers, fonts
<code>export/</code>	Built artifacts
<code>.papyrus/</code>	Incremental caches

Always run from the book root, or pass `-d / --dir`. Override where artifacts are written with `-e / --export` (default: `<book>/export`):

```
papyrus doctor -d /path/to/book
papyrus build --dir /path/to/book
papyrus build:html -d /path/to/book -e /path/to/out
```

Configuration

All settings live in `papyrus.php`, which returns a PHP array.

Identity

```
return [  
  'title' => 'My Book',  
  'subtitle' => 'A short subtitle',  
  'author' => 'Author Name',  
  'themes' => ['light', 'dark'],  
];
```

`themes` lists PDF theme names. Each needs `assets/theme-{name}.html`.

The export filename slug comes from the title (lowercased, hyphenated). This handbook is *The Papyrus Handbook*, so outputs look like `the-papyrus-handbook-light.pdf`.

Page size and margins

```
'document' => [  
  'size' => 'crown-quarto',  
  'margin_left' => 27,  
  'margin_right' => 27,  
  'margin_top' => 14,  
  'margin_bottom' => 14,  
],
```

List presets with **papyrus sizes**. Custom trim:

```
'document' => [  
  'format' => [152.4, 228.6], // width_mm, height_mm  
],
```

Table of contents

```
'toc' => [  
  'h1' => 0,  
  'h2' => 0,  
  'h3' => 1,  
],
```

Values follow mPDF **h2toc** conventions used by Papyrus (**0** include, **1** exclude).

Cover images

```
'cover' => [  
  'image' => 'cover.png',  
  'light' => 'cover-light.png',  
  'dark' => 'cover-dark.png',  
],
```

Paths are relative to **assets/**. Missing per-theme covers fall back to **cover.image**.

Running header

```
'header' => [  
  'style' => 'font-style: italic; text-align: right; border-  
bottom: solid 1px #808080;',  
],
```

Pretoc chapters suppress folios differently from body chapters.

Fonts

Declare faces under **assets/fonts/** and optional **script routing** (PDF only). First matching **script** rule wins.

```
'fonts' => [  
  'default' => 'librelibertine', // optional; else first face,  
  else mPDF default  
  'faces' => [  
    [  
      'name' => 'librelibertine',  
      'regular' => 'LinLibertine_R.ttf',  
      'bold' => 'LinLibertine_RB.ttf',  
      'italic' => 'LinLibertine_RI.ttf',  
      'bold_italic' => 'LinLibertine_RBI.ttf',  
    ],  
    [  
      'name' => 'notosansbengali',  
      'regular' => 'NotoSansBengali-Regular.ttf',  
      'otl' => true, // OpenType layout – needed for  
complex scripts  
    ],  
  ],  
  'script' => [  
    // When mPDF tags a run as Bengali, use this face instead  
of the default  
    ['match' => ['bn', 'ben', 'bengali'], 'face' =>  
'notosansbengali'],  
  ],  
],
```

Faces

Each face needs a **name** and a **regular** file that exists under **assets/fonts/**. Optional keys: **bold**, **italic**, **bold_italic**, and **otl** (sets mPDF **useOTL** when true — common for code fonts and Indic / Arabic scripts).

Papyrus merges these into mPDF's **fontdata** when building PDF (**FontRegistry** → **MpdfFactory**).

Script rules

fonts.script is a list of { **match**, **face** } rules used only for **PDF**. They do not change EPUB or HTML typography.

When any applicable rule exists, Papyrus turns on mPDF's **autoScriptToLang** and **autoLangToFont**, and installs a custom **languageToFont** resolver (**ScriptLanguageToFont**). mPDF then:

1. Detects script runs in the HTML (e.g. Bengali glyphs).
2. Tags them with a language / script code (e.g. **bn**, or ***-beng**).
3. Asks Papyrus which face to use; the first rule whose **match** list hits that code wins.

match aliases are lowercased. Built-in aliases include:

Aliases you can list	Resolves as
bn , ben , beng , bengali	Bengali

Aliases you can list	Resolves as
<code>ar</code> , <code>arab</code> , <code>arabic</code>	Arabic
<code>hi</code> , <code>hin</code> , <code>deva</code> , <code>devanagari</code> , <code>hindi</code>	Devanagari
any 4-letter ISO script code	that script as-is

The `face` string must be a name from `fonts.faces` and that face must actually load (regular file present). Rules pointing at a missing face are skipped (`FontRegistry::applicableScriptRules()`). Unmatched languages fall through to mPDF's built-in `LanguageToFont`.

Example: body text in Latin uses `librelibertine`; a paragraph containing বন্ধন is auto-tagged Bengali and rendered with `notosansbengali` if that face is registered.

Notice: Without both a registered face and a matching `script` rule, non-Latin runs often show as tofu (missing glyphs) in the default font.

Break level

```
'breakLevel' => 2,
```

Automatic page breaks before headings (**1** = H1, **2** = H1 and H2). Explicit **[break]** markers always insert a break.

CommonMark hook

```
'configure_commonmark' => [  
  // callable(s) receiving the Environment  
],
```

Writing content

Chapters are Markdown under **content/**, loaded in natural filename order (**00-...**, **01-...**, **10-...**).

Front matter

```
---
title: My chapter
pretoc: true
---

Body starts here.
```

Key	Meaning
<code>title</code>	Chapter title for EPUB / internal use
<code>pretoc</code>	<code>true</code> places the chapter before the PDF TOC

Markdown features

CommonMark + GFM, fenced code highlighting, and book extras.

Emphasis and code

```
Hello, world.

`inline code`

```php
$user = User::factory()->create();
```
```

Callouts

```
> {notice} Something helpful.
> {warning} Something risky.
```

```
::: note
A note aside.
:::

::: warning
A warning aside.
:::
```

Page breaks

```
[break]
```

Mermaid

```
```mermaid
flowchart TD
 A[Markdown] --> B[Build]
```
```

Enable in `papyrus.php`:

```
'mermaid' => [  
  'enabled' => true,  
  'format' => 'svg',  
  'theme' => 'auto',  
  'max_width_mm' => 130,  
],
```

Requires **mmdc** on **PATH**. Diagrams cache under **.papyrus/cache/mermaid**.

PHP fence linting

```
papyrus lint  
papyrus lint --fix
```

Themes and assets

Everything visual lives under `assets/`. New projects from `papyrus init` ship the same PDF theme used by *The Filament Playbook* (Libre Libertine body, Times headings, OxProto code, github-gist highlight colors, notice/tip/caution asides) plus matching font files under `assets/fonts/`.

PDF themes

For each name in `themes`, provide `assets/theme-{name}.html`. Split the theme on the TOC marker:

```
<!-- PAPYRUS:TOC -->
```

Papyrus writes pretoc chapters, then the TOC, then body chapters around that marker. A legacy `<!-- IBIS:TOC -->` marker is still recognized after `migrate-ibis`.

Typical theme responsibilities: `@page` / typography, code blocks, callout colors, title page, and running header styles (often paired with `header.style` in config).

HTML theme

`assets/theme-html.html` is a full HTML document with placeholders `{{ $title }}`, `{{ $subtitle }}`, `{{ $author }}`, and `{{ $body }}`. The default template mirrors the light PDF theme (same typefaces, colors, code blocks, and callouts), with a fixed moon/sun icon toggle that switches to the dark PDF palette. The reading column is **52em** wide (a bit wider than the PDF measure). `@font-face` rules in the theme use `../assets/fonts/`; at build time Papyrus rewrites those URLs relative to the HTML output directory (so `-e docs` still finds the book's fonts).

```
papyrus build:html
```

EPUB styles

`assets/style.css` (and optional `highlight.codeblock.min.css`) ship inside the EPUB. Prefer simple `pre/code` rules for e-ink readers.

Covers and fonts

Place cover images and font files under `assets/` (fonts usually in `assets/fonts/`). Reference them from `cover` and `fonts.faces` in `papyrus.php`.

Building exports

Validate first, then build.

```
papyrus doctor  
papyrus build
```

build runs every configured PDF theme, then EPUB, HTML, and any enabled KDP outputs.

Individual formats

```
papyrus build:pdf
papyrus build:pdf --theme light
papyrus build:pdf --theme light,dark --parallel
papyrus build:epub
papyrus build:html
papyrus build:sample
```

| Command | Output (slug from title) |
|---------------------------|---|
| <code>build:pdf</code> | <code>export/<slug>-<theme>.pdf</code> |
| <code>build:epub</code> | <code>export/<slug>.epub</code> |
| <code>build:html</code> | <code>export/<slug>.html</code> |
| <code>build:sample</code> | <code>export/sample-<slug>-<theme>.pdf</code> |

Use `-e` / `--export` to write somewhere other than `export/` (for example this repo's handbook docs: `-d examples/the-papyrus-handbook -e docs`).

`--parallel` / `-p` builds each PDF theme in its own process.

Watch mode

```
papyrus watch  
papyrus watch --interval 3
```

Caching

Chapter HTML caches under `.papyrus/cache/markdown`. Mermaid figures use `.papyrus/cache/mermaid`. Delete `.papyrus/` for a cold rebuild.

Vendor noise

mPDF and the EPUB zipper sometimes emit PHP warnings. Papyrus collects third-party notices and only prints them with `-v` / `--verbose`.

Samples and KDP

Sample PDF

Carve page ranges from a full theme PDF:

```
'sample' => [  
  'ranges' => [  
    ['from' => 1, 'to' => 4],  
    ['from' => 10, 'to' => 12],  
  ],  
,  
  'sample_notice' => 'This is a sample from My Book.',
```

```
papyrus build:sample --theme light
```

Pages are 1-based inclusive ranges against the finished PDF.

Amazon KDP

```
'kdp' => [  
  'ebook' => [  
    'enabled' => true,  
    'cover' => 'cover-ebook.jpg',  
  ],  
  'print' => [  
    'enabled' => true,  
    'bleed_mm' => 3,  
    'margin_preset' => 'recommended',  
    'paper' => 'white',  
  ],  
  'metadata' => [  
    'description' => 'Bookstore blurb...',  
    'keywords' => ['laravel', 'filament'],  
    'language' => 'en',  
  ],  
],
```

```
papyrus kdp  
papyrus kdp:ebook  
papyrus kdp:print --theme light  
papyrus kdp:cover  
papyrus kdp:metadata
```

| Command | Typical artifact |
|---------------------------|--|
| <code>kdp:ebook</code> | <code>export/<slug>-kdp.epub</code> |
| <code>kdp:print</code> | <code>export/<slug>-kdp-print.pdf</code> |
| <code>kdp:cover</code> | cover assets under <code>export/</code> |
| <code>kdp:metadata</code> | <code>export/<slug>-kdp-metadata.json</code> |

Confirm trim and margins with `papyrus sizes` and Amazon's current KDP print specs before uploading.

Migration and CI

Migrating from ibis-next

```
papyrus migrate-ibis
papyrus migrate-ibis -d /path/to/book
```

This writes `papyrus.php` and updates theme TOC markers to `<!-- PAPYRUS:TOC -->`. After migration:

1. Remove `hi-folks/ibis-next` and any Composer patches for `ibis/mPDF`.
2. Point scripts at `papyrus` (`build`, `build:pdf`, ...).
3. Run `papyrus doctor` and a full `papyrus build`.

Continuous integration

Copy the stub workflow from the Papyrus package:

```
stubs/github/workflows/book-build.yml
```

A typical book job installs PHP with `dom`, `gd`, `mbstring`, `zip`, `zlib`, runs `composer install`, `papyrus doctor`, optional `papyrus lint`, then `composer build`.

Programmatic use

```
use Milon\Papyrus\Config\Project;  
  
$project = Project::load($bookDir);  
$book =  
$project->bookConverter()->convertDirectory($project->contentDir)  
;
```

Checklist before release

1. `papyrus doctor` is clean
2. `papyrus build` succeeds for every theme
3. Sample ranges still make sense after reflows
4. KDP EPUB opens in Kindle Previewer; print PDF matches trim/bleed
5. HTML export readable on phone and desktop
6. Pin a Papyrus version in the book's `composer.json`

Command reference

| Command | Purpose |
|---------------------------|--|
| <code>init</code> | Scaffold <code>papyrus.php</code> , <code>content/</code> , <code>assets/</code> |
| <code>doctor</code> | Validate config, paths, and optional Mermaid |
| <code>build</code> | All PDF themes, EPUB, HTML, enabled KDP |
| <code>build:pdf</code> | PDF themes (<code>--theme</code> , <code>--parallel</code>) |
| <code>build:epub</code> | EPUB3 |
| <code>build:html</code> | Single-file HTML |
| <code>build:sample</code> | Sample PDF from <code>sample.ranges</code> |
| <code>kdp</code> | All enabled KDP outputs |
| <code>kdp:ebook</code> | Kindle EPUB |
| <code>kdp:print</code> | Print interior PDF |
| <code>kdp:cover</code> | Cover assets under <code>export/</code> |

| Command | Purpose |
|---------------------------|--|
| <code>kdp:metadata</code> | Metadata JSON sidecar |
| <code>sizes</code> | List page-size presets |
| <code>migrate-ibis</code> | <code>ibis.php</code> → <code>papyrus.php</code> |
| <code>lint</code> | Lint PHP fences (<code>--fix</code>) |
| <code>watch</code> | Rebuild on changes (<code>--interval</code>) |

Common options:

- `-d` / `--dir` — book root (default: current directory)
- `-e` / `--export` — override export directory (default: `<book>/export`)
- `-v` / `--verbose` — include third-party vendor notices

Packagist: [milon/papyrus](#). Source: github.com/milon/papyrus.